Mentimeter

Laborator 9

Polimorfim si clase

Mentimeter

Polimorfim

- interfata comuna (set de operatii) pentru mai multe tipuri
- polimorfism parametric
- polimorfism ad-hoc

Mentimeter

Polimorfism parametric

- exista functii care au acceasi implementare indiferent de tipul datelor stocate
- exemplu: lungimea unei liste nu depinde de tipul elementelor (Int/Char/altceva)
- => putem folosi un tip generic a pentru a caracteriza elementele

Mentimeter

```
*Main> :t map
map :: (a -> b) -> [a] -> [b]
*Main> :t filter
filter :: (a -> Bool) -> [a] -> [a]
*Main> :t zipWith
zipWith :: (a -> b -> c) -> [a] -> [b] -> [c]
*Main> :t (,)
(,) :: a -> b -> (a, b)
*Main> :t []
[] :: [a]
```

Exemple

Mentimeter

Polimorfism ad-hoc

- folosim tipuri generice, dar avem nevoie de metode specifice (ex: ==, show)
- => adaugam constrangeri de tip (ex: Eq a =>)

Mentimeter

```haskell
data Pair a = Pair a a deriving Eq

remove :: Eq a => [Pair a] -> Pair a -> [Pair a]
remove all to_remove = filter (\p -> p /= to_remove) all
```

Exemplu (am nevoie de Eq a si Eq (Pair a) pentru a folosi /=)

Mentimeter

Aceeasi implementare, comportament diferit

Functia remove are aceeasi implementare, dar comportamentul ei depinde de implementarea lui

/=

Mentimeter

Clase

- in Haskell sunt asemanatoare cu interfetele din Java (**interfata, nu clasa**)
- un set de functii care definesc o interfata/comportament unitar pentru un tip de date

Mentimeter

```
class Eq a where                                                    # Source
```

The `Eq` class defines equality (`==`) and inequality (`/=`). All the basic datatypes exported by the `Prelude` are instances of `Eq`, and `Eq` may be derived for any datatype whose constituents are also instances of `Eq`.

The Haskell Report defines no laws for `Eq`. However, instances are encouraged to follow these properties:

Reflexivity
`x == x = True`
Symmetry
`x == y = y == x`
Transitivity
if `x == y && y == z = True`, then `x == z = True`
Extensionality
if `x == y = True` and `f` is a function whose return type is an instance of `Eq`, then `f x == f y = True`
Negation
`x /= y = not (x == y)`

Minimal complete definition: either `==` or `/=`.

Minimal complete definition

```
(==) | (/=)
```

Methods

```
(==) :: a -> a -> Bool    infix 4                                   # Source
```

```
(/=) :: a -> a -> Bool    infix 4                                   # Source
```

Exemplu Eq a: 2 metode ==, /= (**numai una este necesara**)

Mentimeter

Minimal complete definition

```
compare | (<=)
```

Methods

```
compare :: a -> a -> Ordering                    # Source

(<) :: a -> a -> Bool    | infix 4 |             # Source

(<=) :: a -> a -> Bool   | infix 4 |             # Source

(>) :: a -> a -> Bool    | infix 4 |             # Source

(>=) :: a -> a -> Bool   | infix 4 |             # Source

max :: a -> a -> a                               # Source

min :: a -> a -> a                               # Source
```

Ord a vs Eq a: pot sa si ordonez elementele, **nu numai sa verific daca sunt egale sau nu**



Mentimeter

Cum putem sa inrolam un tip la o clasa?

- crearea unei instance si implementarea metodelor necesare
- sau folosind deriving (disponibil **numai pentru unele clase default**)

Mentimeter

```haskell
data Tree a = Empty | Node (Tree a) a (Tree a)

{-
    Vreau sa afisez valoarea din nod
    => ma pot folosi de functia show, dar tipul valorii (a) trebuie sa o implementeze
    => constrangere: Show a =>
-}
instance Show a => Show (Tree a) where
    show Empty = "Null"
    show (Node left value right) = (show left) ++ (show value) ++ (show right)
```

Exemplu

Mentimeter

```haskell
data Pair a b = Pair a [b]         -- cheie: [valori]
data Dict a b = Dict [Pair a [b]] -- un dictionar are o secventa de perechi (cheie, valori)

{-
    Nu imi pasa de ordinea in care sunt salvate cheile atat timp cat sunt egale
    => pot sa le sortez inainte sa le compar

    1) Folosesc == pe cele 2 liste de perechi(cu tipul Pair a [b])
       => am nevoie de Eq (Pair a [b])
    2) Folosesc sort pe liste (acelasi tip Pair a [b]) => imi trebuie Ord (Pair a [b])

    Daca un tip apartine lui Ord atunci apartine si lui Eq
    => (1) + (2) = avem nevoie numai de Ord (Pair a [b]) care are nevoie de Ord a si Ord b
-}

instance (Ord a, Ord b) => Eq (Dict a b) where
    Dict dict1 == Dict dict2 = (sort dict1) == (sort dict2)
{-
    Pentru a aplica sort/== pe perechi o sa trebuiasca sa implementam metodele necesare:
    1) == pentru Eq
    2) <=, compare pentru Ord
-}

instance (Eq a, Eq b) => Eq (Pair a [b]) where
    Pair a1 b1 == Pair a2 b2 = (a1 == a2) && (b1 == b2)

instance (Ord a, Ord b) => Ord (Pair a [b]) where
    Pair a1 _ <= Pair a2 _ = a1 <= a2
    compare (Pair a1 _) (Pair a2 _) = compare a1 a2
```

Exemplu mai fancy

Mentimeter

Explicatie: (T1 a) => (T2 (T3 a))

Daca a apartine clasei T1 atunci si (T3 a) apartine clasei T2 (ex: Show a => Show (Tree a): daca a apartine Show atunci si Tree a o sa aprtina)

Mentimeter

Deriving

- implementari default pentru metodele cerute
- ex: la Eq se compara pe rand fiecare camp al componentelor
- **disponibil numai pentru Ord, Enum, Bounded, Show, Read**

Mentimeter

Extinderea de clase

- Haskell permite ca o clasa sa extinda o alta clasa
- ex: nu putem vorbi de ordonarea elementelor dintr-un tip daca nu le putem verifica egalitatea

Mentimeter

```haskell
class  (Eq a) => Ord a  where
    compare              :: a -> a -> Ordering
    (<), (<=), (>), (>=) :: a -> a -> Bool
    max, min             :: a -> a -> a

    compare x y = if x == y then EQ
                  else if x <= y then LT
                  else GT

    x <  y = case compare x y of { LT -> True;  _ -> False }
    x <= y = case compare x y of { GT -> False; _ -> True }
    x >  y = case compare x y of { GT -> True;  _ -> False }
    x >= y = case compare x y of { LT -> False; _ -> True }

    max x y = if x <= y then y else x
    min x y = if x <= y then x else y
```

Exemplu Ord/Eq



Mentimeter

class (T1 a) => T2 a where

Tipul a este in T2 daca este in T1 si
implementeaza functiile de mai jos

Mentimeter

Membri impliciti

- daca stiu cand 2 elemente sunt egale => stiu cand nu sunt diferite
- pot sa imi definesc /= ca not ==
- si echivalent == ca not /=

Mentimeter

```
class  Eq a  where
    (==), (/=)              :: a -> a -> Bool
    x /= y                  = not (x == y)
    x == y                  = not (x /= y)
```

Exemplu Eq

Mentimeter

Explicatie

Trebuie sa implementez **obligatoriu** fie ==,
fie /= (nu neaparat ambele, dar daca vreau
o implementare diferita pot sa le
implementez pe amandoua)

Mentimeter

Num

- tip generic pentru valori numerice
- ex: 5 poate fi un numar intreg(5) sau fractionar(5.0)
- nu alegem un tip concret (ex: Int) pana ce nu ne restrictioneaza contextul

Mentimeter

Alte clase predefinite

- Ord: tipuri ordonabile (<, >, <=, etc)
- Show: tipuri reprezentabile sub forma de String-uri
- Read: inversa Show = tipuri care pot fi citite din String-uri

Mentimeter

Alte clase predefinite 1

- Enum: tipuri care pot fi enumerate (ex: functii folosite pentru [a..b])
- Num: toate tipurile numerice cu operatii aritmetice de baza: +, -, etc
- Integral: toate tipurile intregi (Int, Integral sunt incluse aici si difera prin range-ul acoperit)

Mentimeter

Alte clase predefinite 2

- Fractional: numere reprezentabile prin fractii
- Floating: numere reale cu functii precum sqrt, exp, etc

Mentimeter

```haskell
data List a = EmptyList | Cons a (List a)
data Tree a = EmptyTree | Node (Tree a) a (Tree a)

listMap :: (a -> b) -> List a -> List b
listMap f EmptyList = EmptyList
listMap f (Cons x l) = Cons (f x) (listMap f l)

treeMap :: (a -> b) -> Tree a -> Tree b
treeMap f EmptyTree = EmptyTree
treeMap f (Node left value right) = Node (treeMap f left) (f value) (treeMap f right)
```

Tipuri de ordin superior



Mentimeter

Implementari diferite, dar acelasi antet

- solutie: definirea unei clase comune si instantierea ei diferit in functie de tip
- indiferent de container, map ia o functie (a->b) si transforma elementele din tipul a in tipul b

Mentimeter

```
class Functor dataStructure where
    map :: (a -> b) -> dataStructure a -> dataStructure b
```

Clasa comuna (pot sa adaug orice alte metode vreau sa fie implementate de fiecare tip)

Mentimeter

```haskell
instance Functor List where
    map f EmptyList = EmptyList
    map f (Cons x l) = Cons (f x) (map f l)

instance Functor Map where
    map f EmptyTree = EmptyTree
    map f (Node left value right) = Node (map f left) (f value) (map f right)
```

Instantieri

